

Two Pointers

An interview study guide built from a Socratic-style walkthrough

This guide is built from a ChatGPT dialogue that worked through the Two Pointers technique one proof at a time. The central rule, repeated at almost every step of the source conversation, is this:

Don't memorize "move left" or "move right". Prove that moving that pointer safely **eliminates possibilities** that cannot contain the answer.

Two pointers is not "two variables called `left` and `right`" -- it is a family of techniques that all share one property: at every step, you can look at the current pair of positions and mathematically prove that at least one of them can never be part of a better answer, so you're allowed to permanently move past it. Different problems eliminate possibilities for different reasons, and telling those reasons apart is the actual skill this guide is trying to build.

Roadmap

1. **Opposite-end pointers** -- the elimination proof, Two Sum II, Container With Most Water, and string palindromes.
2. **Same-direction pointers on one sequence** -- Is Subsequence.
3. **Same-direction slow/fast pointers** -- the processed-region pattern: Remove Duplicates, Remove Element, Move Zeroes, Sort Colors.
4. **Sort + fixed pointer(s) + two pointers** -- the kSum family: 3Sum, 3Sum Closest, 3Sum Smaller, 4Sum, and the duplicate-handling rules that trip almost everyone up at least once.
5. **Merge-style pointers** -- one pointer per sequence.
6. **Hard problems requiring their own proof** -- Trapping Rain Water, Boats to Save People, Valid Triangle Number, Bag of Tokens, Meeting Scheduler.
7. **Recognition** -- how to tell, from the problem statement alone, which pattern (if any) applies -- and when two pointers is the *wrong* tool.
8. **Decision guide, recognition drills, and interview Q&A.**

What this guide deliberately skips

The source chat covered fast/slow pointers on a linked list (finding the middle, cycle detection) and fixed-gap pointers (kth node from the end) only at the recognition level -- the user had already covered both in the Linked List guide, and the chat explicitly deferred to that material ("i learned this in linked list already, continue"). This guide does the same: Part 7's recognition table references both patterns so the full picture is in one place, but the derivations and code live in the **Linked List** guide, not here. Two-pointer problems that happen to sit *on an array* (not a linked list) are covered in full below.

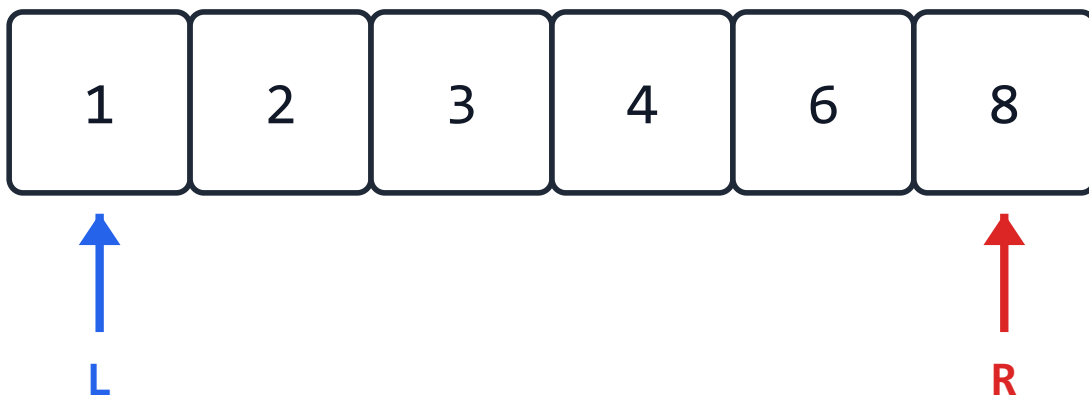
Part 1: Opposite-End Pointers -- The Elimination Proof

1.1 The core idea, derived from scratch

Given a sorted array, place `left = 0` and `right = n - 1`. At every step look at `nums[left] + nums[right]` against a target sum. There are only three cases:

- **sum too small** -- `nums[right]` is already the *largest* available partner for `nums[left]`. If even the largest partner isn't enough, `nums[left]` can never reach the target with **any** partner. So `nums[left]` is eliminated -- move `left`.
- **sum too large** -- symmetric argument: `nums[left]` is already the *smallest* available partner for `nums[right]`, so `nums[right]` is eliminated -- move `right`.
- **sum equal** -- done.

$1 + 8 = 9 < 10 \rightarrow$ every pair using 1
is ≤ 9



1 eliminated entirely -> $2 + 8 = 10$,
found



The mental model to internalize is **not** "sum small -> left++". It's:

Which pointer is sitting on a value that I can prove can never be part of the answer? That's the pointer we eliminate.

This only works because the array is **sorted** -- sorting gives a monotonic relationship (`nums[left]` only ever gets bigger as `left` increases, `nums[right]` only ever gets smaller as `right` decreases), which is exactly what makes "even the extreme partner isn't enough" a valid proof of impossibility. Without sorting, there's no guarantee that a smaller value doesn't exist somewhere in the middle of the array.

1.2 Worked correction: eliminating the *right* value (a mistake caught live in the source chat)

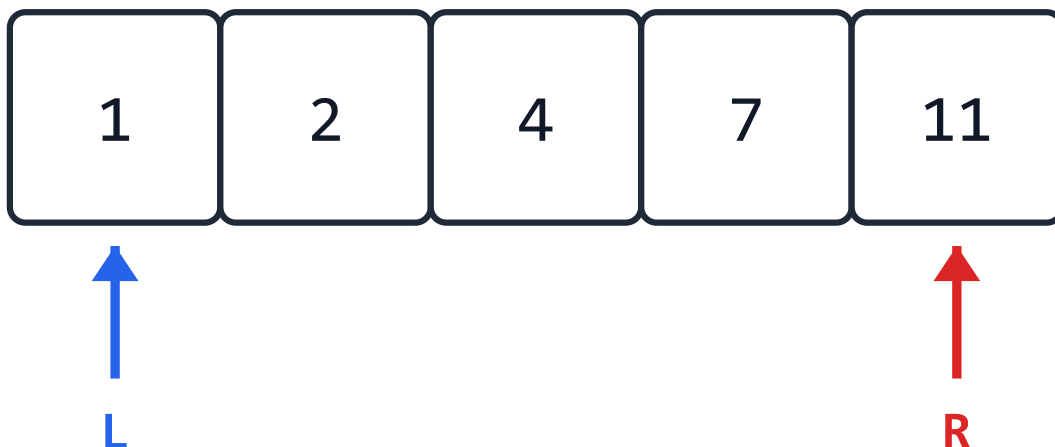
A correction worked through live in the source chat

Given `nums = [1, 2, 4, 7, 11]`, `target = 13`, with `L=0`, `R=4`: $1 + 11 = 12 < 13$.

The first instinct was: "**R, eliminate 11.**" That's wrong, and the reasoning for *why* it's wrong is the entire point of this pattern. Eliminating `11` would require proving `11` can never be part of a valid pair. But that's false here -- `11` could still pair with something smaller than `1` (there just isn't anything smaller than `1` in *this* array, but the pointer-movement rule has to be correct in general, not just for this instance).

`nums = [1, 2, 4, 7, 11]`, `target = 13`

$$1 + 11 = 12 < 13$$



WRONG first answer: "move R, eliminate 11"

unsafe: 11 might still pair with something smaller than 1 (nothing here is), so 11 can't be ruled out

The actual proof only ever targets the pointer whose current value is the *limiting* one:

- sum too small -> the largest possible partner (`nums[right]`) still wasn't enough -> `nums[left]` is impossible -> move **left**.
- sum too large -> the smallest possible partner (`nums[left]`) was already too much -> `nums[right]` is impossible -> move **right**.

Here the sum is too *small*, so the correct move is `left += 1` (eliminate `1`, not `11`). A quick sanity check confirms the wrong move would actually break the algorithm:

- **correct_move**: `L++` (eliminate 1)
- **wrong_move_fails**: `True`

CORRECT: move L, eliminate 1

11 is already the largest partner:
 $1+11=12 < 13$, so 1 can never reach target
-- only 1 is provably dead

after the correct move (`L++`):



1.3 Two Sum II -- Input Array Is Sorted (LC 167)

Recognition: sorted array + find a pair with an exact target sum -> opposite ends.

Invariant: everything left of `left` and right of `right` has already been proven impossible; the answer, if it exists, is still somewhere in `[left, right]`.

```
def two_sum_sorted(numbers: List[int], target: int) → List[int]:
    """LC 167. Two Sum II - Input Array Is Sorted.

    left/right represent: 'the smallest still-live value' and 'the
    largest still-live value'. At every step exactly one of them can be
    proven impossible and is eliminated.
    """
    left, right = 0, len(numbers) - 1
    while left < right:
        total = numbers[left] + numbers[right]
        if total == target:
            return [left + 1, right + 1]
        if total < target:
            left += 1
        else:
            right -= 1
    return []
```

Complexity: $O(n)$ time -- `left` only ever increases, `right` only ever decreases, so together they take at most n total steps -- $O(1)$ extra space.

Full step-by-step trace (`nums=[1,2,4,7,11]` , `target=13` , after the correction above):

left	right	nums[L]	nums[R]	sum
0	4	1	11	12
1	4	2	11	13

Part 2: Container With Most Water -- A Different Kind of Proof

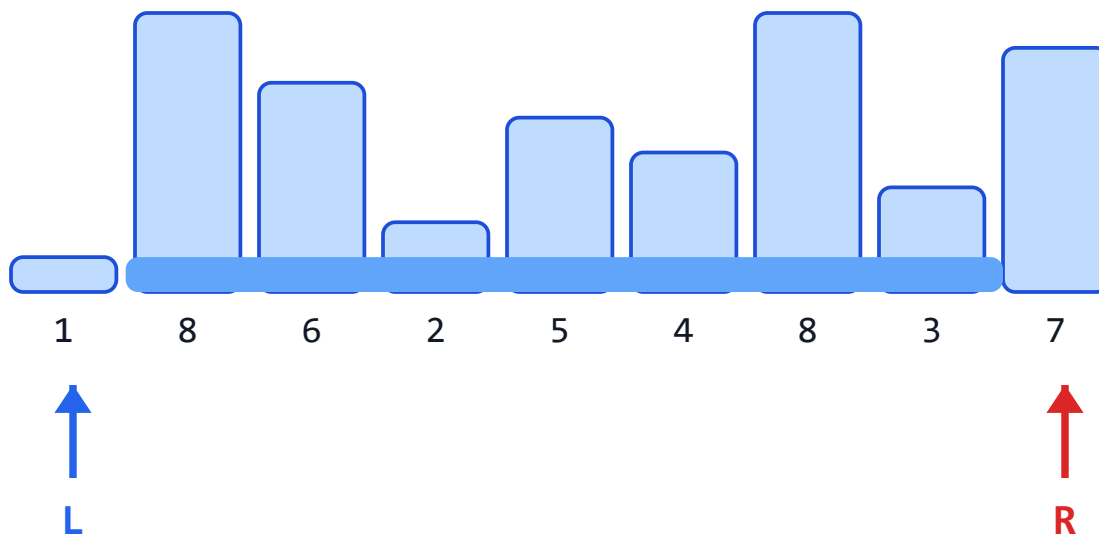
2.1 Why this problem is special

Two Sum II eliminates a pointer because its *value* can be proven impossible for a target. Container With Most Water eliminates a pointer for a completely different reason: not "this value can never reach the target," but **"keeping this pointer can never produce a better answer than what moving it might find."**

Given `height`, the area between `left` and `right` is:

```
area = min(height[left], height[right]) * (right - left)
```

`height=[1,8,6,2,5,4,8,3,7]` `area=min(1,7)x8=8`



2.2 The proof, derived step by step

Suppose `height[left] = 1` and `height[right] = 7`. The current area is `1 x width`. What happens if we move `right` instead of `left`?

- The width shrinks.
- `left` is still height `1`, so the new area is still capped at `1 x (smaller width)`, which is strictly **less** than the current area.

So moving the *taller* side can never produce a better answer -- it's mathematically ruled out, not just unlikely. Moving the *shorter* side, on the other hand, might reveal a taller wall and raise the limiting height. That asymmetry is the entire proof:

`height[L]=1 < height[R]=7 -> which side is safe to move?`

Move R (unsafe)

L stays at height 1, so new area =
`min(1,h) x smaller_width <= 1 x smaller_width < 1 x current_width`
 -> can never beat the current area

Move L (the only useful move)

R stays at height 7, but a taller left wall MIGHT appear, which could raise the limiting height
 -> only this move has a CHANCE to discover a better area

Keep the shorter side -> impossible to improve while width shrinks. Move the shorter side -> only this move gives a chance to discover a taller limiting height.

When `height[left] == height[right]`, both sides are equally limiting, so either pointer is safe to move -- moving either one still shrinks the width while both heights are already used up as the answer for this exact pair, so neither can ever be reused at a better width again.

2.3 Code

```
def max_area(height: List[int]) → int:
    """LC 11. Container With Most Water.

    Not an elimination-by-value proof like Two Sum -- an elimination-by-
    optimization proof. Keeping the shorter side while shrinking width can
    never beat the current area, so only the shorter side is worth moving.
    """
    left, right = 0, len(height) - 1
    best = 0
    while left < right:
        width = right - left
        area = min(height[left], height[right]) * width
        best = max(best, area)
        if height[left] < height[right]:
            left += 1
        elif height[right] < height[left]:
            right -= 1
        else:
            # equal heights: either side is safe to move (both are limiting)
            left += 1
            right -= 1
    return best
```

Complexity: $O(n)$ time (`left` / `right` together take at most `n` steps), $O(1)$ space.

Full trace on `height=[1,8,6,2,5,4,8,3,7]` :

left	right	h[L]	h[R]	area	best
0	8	1	7	8	8
1	8	8	7	49	49
1	7	8	3	18	49
1	6	8	8	40	49
2	5	6	4	12	49
2	4	6	5	10	49
2	3	6	2	2	49

The `best` column shows the running maximum reached at each step; it tops out at `49` and `sol.max_area` returns exactly that value.

Part 3: Opposite-End String Pointers -- Skip, Then Compare

3.1 Valid Palindrome (LC 125)

A third reason opposite-end pointers can move: **the position has already been fully processed and there's nothing left to decide.**

skip non-alphanumeric characters, compare case-insensitively



The invariant while scanning inward: every pair of meaningful characters compared so far matches. We don't need to build a cleaned copy of the string -- `left` and `right` simply skip non-alphanumeric characters in place, then compare case-insensitively.

```
while left < right:
    skip non-alphanumeric at left
    skip non-alphanumeric at right
    compare normalized characters
    mismatch → not a palindrome, stop immediately
    match → move both inward
```

The moment a mismatch is found, we can return immediately -- a palindrome requires **every** mirrored pair to match, so one violation is proof enough, and there is no need to inspect the rest.

```
def is_palindrome(s: str) → bool:
    """LC 125. Valid Palindrome.

    Skip non-alphanumeric characters in place; compare case-insensitively.
    No need to build a cleaned copy of the string first.
    """
    left, right = 0, len(s) - 1
    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1
        if s[left].lower() ≠ s[right].lower():
            return False
        left += 1
        right -= 1
    return True
```

3.2 Valid Palindrome II (LC 680)

Same opposite-end scan, but the first mismatch is allowed **one** deletion. At the first mismatching pair, try skipping the left character *or* the right character, and check whether either resulting substring is a clean palindrome (no more deletions allowed once you've used the one you get):

```
def valid_palindrome_ii(s: str) → bool:
    """LC 680. Valid Palindrome II.

    Same opposite-end scan as LC 125, but the first mismatch is allowed
    one deletion: try skipping the left character OR the right character
    and check whether either resulting substring is a clean palindrome.
    """
    def is_pal_range(lo: int, hi: int) → bool:
        while lo < hi:
            if s[lo] ≠ s[hi]:
                return False
            lo += 1
            hi -= 1
        return True

    left, right = 0, len(s) - 1
    while left < right:
        if s[left] ≠ s[right]:
            skip_left = is_pal_range(left + 1, right)
            skip_right = is_pal_range(left, right - 1)
            return skip_left or skip_right
        left += 1
        right -= 1
    return True
```

This still relies on the same opposite-end elimination proof -- once you fix which single character to drop, the remaining range is checked with the exact same "compare inward, mismatch -> false" logic as 3.1.

3.3 Reverse String (LC 344)

The simplest possible opposite-end pointer problem -- no elimination proof is even needed here, since every position must be visited exactly once; `left / right` just swap inward until they meet:

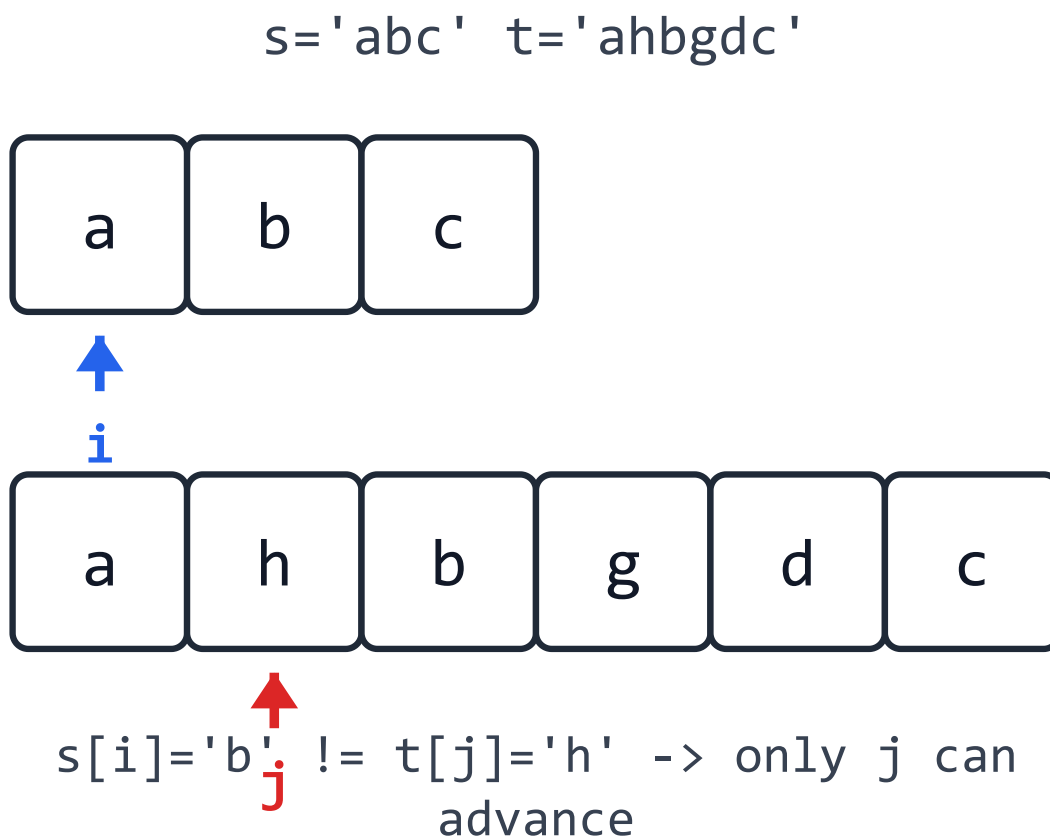
```
def reverse_string(s: List[str]) → None:
    """LC 344. Reverse String (in place). Textbook opposite-end swap."""
    left, right = 0, len(s) - 1
    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1
```

Part 4: Same-Direction Pointers on Two Sequences

4.1 Is Subsequence (LC 392)

This is the first pattern where the two pointers move in the **same** direction, and represent genuinely different things:

- `i` -- the next character of `s` we still need to match.
- `j` -- a scanner walking through `t`.



If `s[i] == t[j]`, we've matched the character we needed -- advance **both**. If they differ, we can't advance `i` (we still haven't matched it), but we're always allowed to advance `j`, because we're permitted to delete characters from `t`.

```
s[i] == t[j] → matched      → i += 1, j += 1
s[i] != t[j] → skip t[j]   → j += 1
```

The invariant: `s[:i]` has already been matched, in order, inside the prefix of `t` scanned so far. Success is `i == len(s)` -- once every character of `s` has been consumed, `s` is a subsequence of

`t`.

```
def is_subsequence(s: str, t: str) → bool:
    """LC 392. Is Subsequence.

    i = next character of s still needed. j = scanner over t. i only
    advances on a match; j always advances, because we're allowed to
    skip characters of t.
    """
    i = j = 0
    while i < len(s) and j < len(t):
        if s[i] == t[j]:
            i += 1
        j += 1
    return i == len(s)
```

Complexity: $O(\text{len}(t))$ -- `j` scans `t` at most once, and `i` can never advance more times than `j` does.

Notice how this differs from opposite-end elimination: neither pointer is being proven "impossible." Instead, `j` is always safe to advance because skipping a character of `t` never loses information we still need, and `i` only advances when it has concrete proof of a match.

Part 5: Same-Direction Slow/Fast Pointers -- Processed Region | Scanner

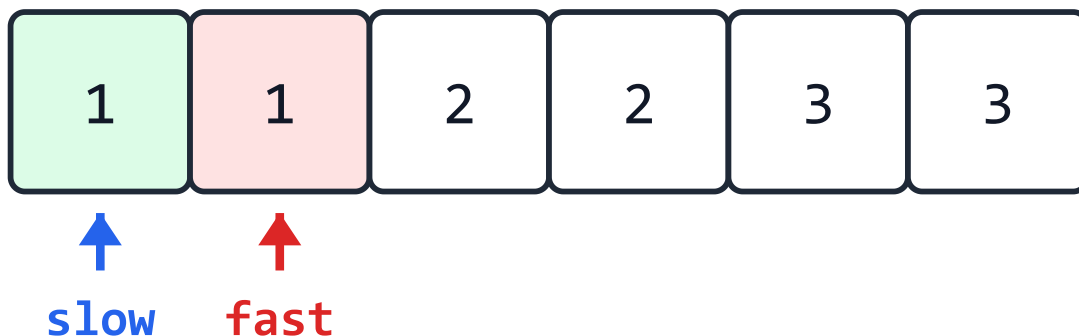
5.1 The pattern, in general

This is a third fundamentally different reason for pointer movement, and it has nothing to do with sorted-pair elimination:

- `slow` -- the boundary of the region we've already correctly built.
- `fast` -- a scanner searching the rest of the array for something that belongs in that region.

When `fast` finds something that belongs in the processed region, it gets written at `slow`, and `slow` advances. Otherwise `slow` simply stays put while `fast` keeps scanning. The exact write mechanism (direct assignment vs. a swap) is an implementation detail -- the *idea* is always "processed region | unprocessed scanner."

`fast` sees a DUPLICATE of `nums[slow]` -> `slow` does not move



5.2 Remove Duplicates from Sorted Array (LC 26)

`slow` marks the boundary of the deduplicated-so-far prefix. Because the array is sorted, duplicates are always adjacent, so `fast` only has to compare against the single most-recently-kept value (`nums[slow - 1]`).


```
def remove_duplicates(nums: List[int]) → int:
    """LC 26. Remove Duplicates from Sorted Array.

    slow = boundary of the deduplicated-so-far region. fast scans; when
    fast finds a genuinely new value, it gets written at slow and slow
    advances.
    """
    if not nums:
        return 0
    slow = 1
    for fast in range(1, len(nums)):
        if nums[fast] != nums[slow - 1]:
            nums[slow] = nums[fast]
            slow += 1
    return slow
```

Trace on `[1,1,2,2,3,3]` (columns: slow position, fast index, did we write a new value this step):

slow	fast	wrote?
0	0	[1, 1, 2, 2, 3, 3]
1	1	False
2	2	True
2	3	False
3	4	True
3	5	False

5.3 Remove Duplicates from Sorted Array II (LC 80)

A direct extension: each value is now allowed to appear **at most twice**. The only change is what `fast` compares against -- instead of the single previous kept value, compare against the value two slots back (`nums[slow - 2]`), so a value is rejected only once it would be a *third* consecutive occurrence.

```
def remove_duplicates_ii(nums: List[int]) → int:
    """LC 80. Remove Duplicates from Sorted Array II.

    Same processed-region idea, but a value is allowed to appear at most
    twice, so we compare fast against nums[slow - 2] instead of
    nums[slow - 1].
    """
    if len(nums) ≤ 2:
        return len(nums)
    slow = 2
    for fast in range(2, len(nums)):
        if nums[fast] ≠ nums[slow - 2]:
            nums[slow] = nums[fast]
            slow += 1
    return slow
```

5.4 Remove Element (LC 27)

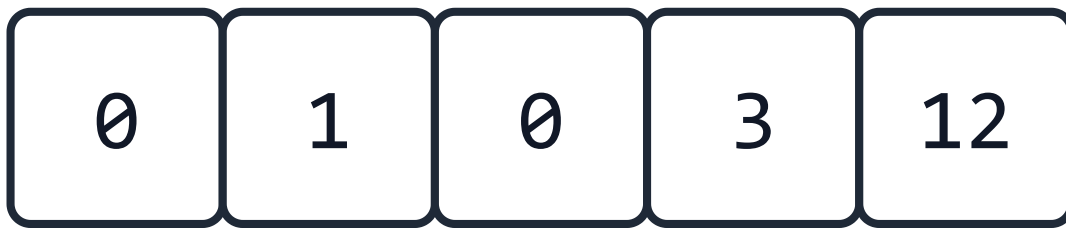
Same processed-region idea, generalized: `fast` looks for anything that is **not** equal to `val`. A swap is one implementation, but the concept is "compact the elements we want to keep toward the front":

```
def remove_element(nums: List[int], val: int) → int:
    """LC 27. Remove Element.

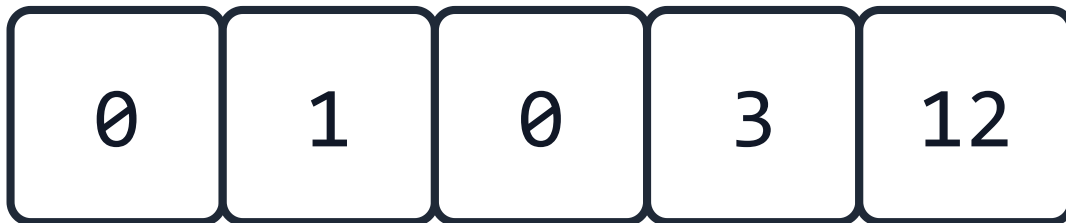
    slow = next destination for a value we want to keep. fast scans and
    writes through slow whenever it finds a value ≠ val. The swap seen
    in some writeups is just one implementation detail, not the idea.
    """
    slow = 0
    for fast in range(len(nums)):
        if nums[fast] ≠ val:
            nums[slow] = nums[fast]
            slow += 1
    return slow
```

5.5 Move Zeroes (LC 283)

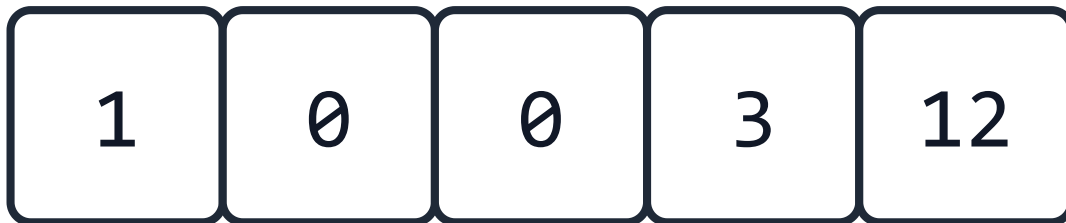
Here "valid" means "non-zero." Once every non-zero element has been placed in front (`slow` positions), there is no need to hunt for the zeroes separately -- everything from `slow` onward simply gets overwritten with `0`, because nothing else is left to place there.



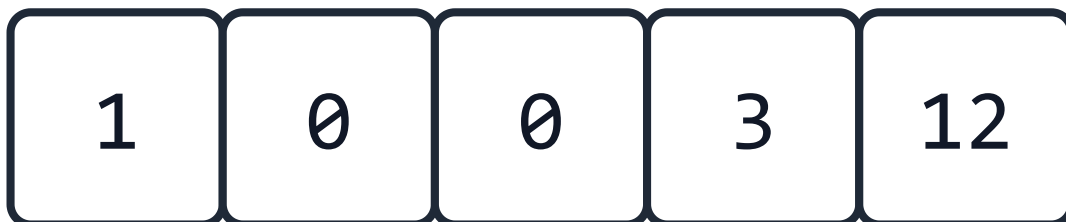
start: fast sees 0 -> slow stays



fast sees 1 -> write at slow,
slow++



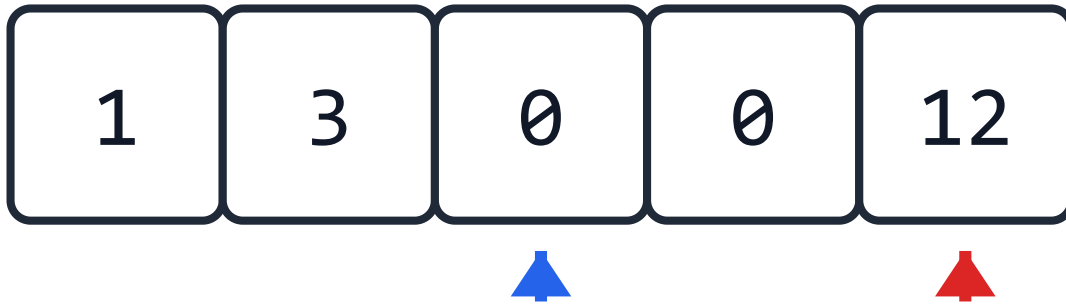
fast sees 0 -> slow stays



fast sees 3 -> write at slow

fast sees 3 -> write at slow, slow++

slow++



fast sees 12 -> write at slow, slow++

```
def move_zeroes(nums: List[int]) -> None:
    """LC 283. Move Zeroes (in place).

    slow marks where the next non-zero belongs. Once every non-zero has
    been placed, everything after slow must be zero -- no separate step
    is needed to search for zeroes, we can just overwrite the tail.
    """
    slow = 0
    for fast in range(len(nums)):
        if nums[fast] != 0:
            nums[slow] = nums[fast]
            slow += 1
    while slow < len(nums):
        nums[slow] = 0
        slow += 1
```

Full step trace on [0,1,0,3,12] (columns: fast index, slow after this step, array state):

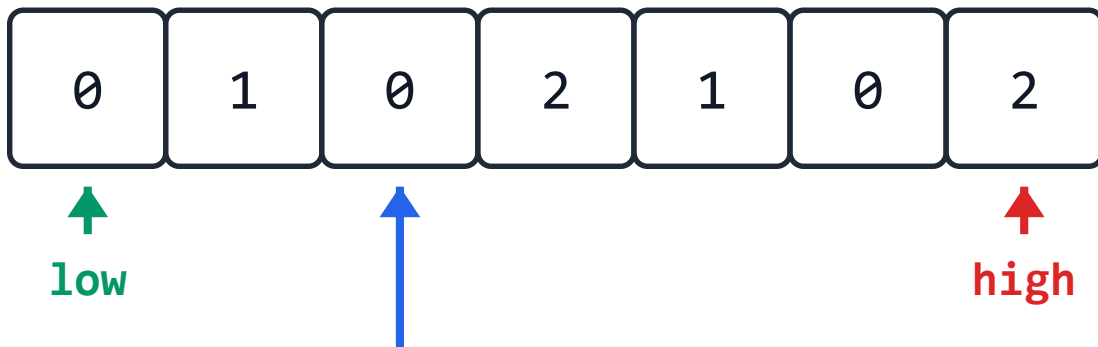
fast	slow	array
0	0	[0, 1, 0, 3, 12]
1	1	[1, 1, 0, 3, 12]
2	1	[1, 1, 0, 3, 12]
3	2	[1, 3, 0, 3, 12]
4	3	[1, 3, 12, 3, 12]

5.6 Sort Colors -- Dutch National Flag (LC 75)

An extension of the same idea to **three** regions instead of two, tracked with three pointers:

```
[ 0 region | 1 region | unknown | 2 region ]
           low      mid      high
```

[0 region][1 region][unknown][2 region] -- mid
scans unknown



mid sees 2: swap **mid** with high, DON'T advance mid (the
swapped-in value is still unchecked)

The one genuinely subtle rule: when **mid** finds a 2, it swaps with **high** and shrinks the "unknown" region from the right -- but **mid** must **not** advance, because the value just swapped in from **high** has never been examined and could itself be a 0, 1, or another 2. (When **mid** finds a 0, the value swapped in from **low** is always either untouched or already known to be a 1 from earlier in the scan, so it's safe to advance both pointers in that branch.)

```
def sort_colors(nums: List[int]) → None:
    """LC 75. Sort Colors (Dutch National Flag, in place).

    Three regions: [0 region][1 region][unknown][2 region], tracked by
    low/mid/high. When mid finds a 2, mid must NOT advance -- the value
    swapped in from high is still unprocessed and needs to be examined.
    """
    low, mid = 0, 0
    high = len(nums) - 1
    while mid ≤ high:
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1
            mid += 1
        elif nums[mid] == 1:
            mid += 1
        else:
            nums[mid], nums[high] = nums[high], nums[mid]
            high -= 1
```

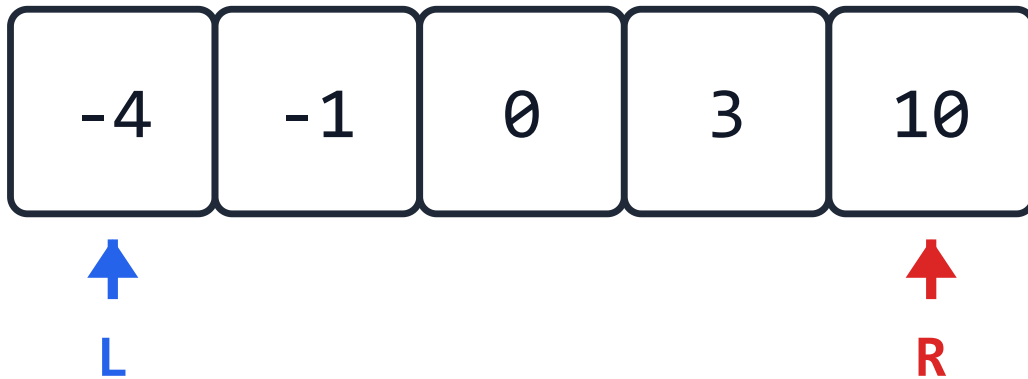
Trace on `[0,1,0,2,1,0,2]` (columns: low, mid, high, array state):

low	mid	high	array
1	1	6	[0, 1, 0, 2, 1, 0, 2]
1	2	6	[0, 1, 0, 2, 1, 0, 2]
2	3	6	[0, 0, 1, 2, 1, 0, 2]
2	3	5	[0, 0, 1, 2, 1, 0, 2]
2	3	4	[0, 0, 1, 0, 1, 2, 2]
3	4	4	[0, 0, 0, 1, 1, 2, 2]
3	5	4	[0, 0, 0, 1, 1, 2, 2]

5.7 Squares of a Sorted Array (LC 977)

A different flavor entirely -- not an elimination proof about a target, but about where the *largest remaining square* can come from. Since the array is sorted but may contain negatives, the biggest square always sits at one of the two ends (the most negative or the most positive value). Fill the answer array from the **back**, always taking whichever end has the larger absolute value.

$|-4| > |10|$? no $\rightarrow$ R gives the bigger square, fill from the BACK



```
def squares_of_sorted_array(nums: List[int]) -> List[int]:
    """LC 977. Squares of a Sorted Array.

    Not an elimination proof about a target -- the biggest square always
    comes from whichever end has the larger absolute value, so we fill
    the answer from the back.
    """
    n = len(nums)
    result = [0] * n
    left, right = 0, n - 1
    for i in range(n - 1, -1, -1):
        if abs(nums[left]) > abs(nums[right]):
            result[i] = nums[left] * nums[left]
            left += 1
        else:
            result[i] = nums[right] * nums[right]
            right -= 1
    return result
```

Part 6: Sort + Fixed Pointer(s) + Two Pointers -- The kSum Family

6.1 Why sorting matters here

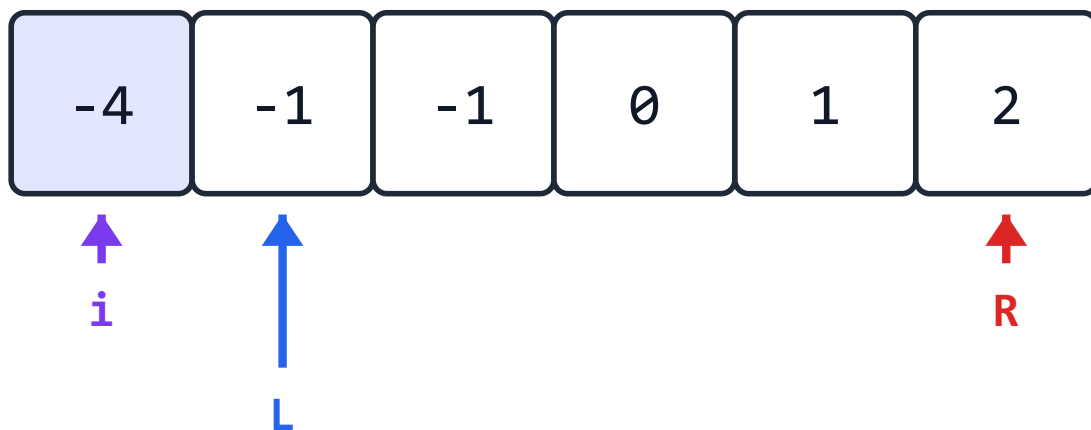
The recurring question in this part of the source chat: **why does sorting matter so much?**

Consider an unsorted array like `[8, 1, 6, 3, 4]` with `8 + 1 = 9 < target`. Could we say "1 is too small, move left"? No -- without sorted order there's no guarantee some smaller value (like a 2) doesn't exist elsewhere in the array. Sorting is what turns "the sum is too small" into a valid *proof* that the current pointer's value is impossible, because it guarantees a monotonic relationship: moving `left` right only ever increases `nums[left]`, and moving `right` left only ever decreases `nums[right]`.

6.2 3Sum (LC 15)

Recognition: find triples -> sort, fix one index `i`, run the exact Two Sum II elimination proof on the rest of the array.

fix `i=-4`, run L/R on the rest: $-4 + -1 + 2 = -3 < 0$




```

sort
fix i
L/R scan the rest:
    sum < 0 → L++    (nums[i]+nums[L] is the smallest achievable
                     with this L, and it's still too small)
    sum > 0 → R--
    sum = 0 → record, then skip duplicates, then move both

```

```

def three_sum(nums: List[int]) → List[List[int]]:
    """LC 15. 3Sum.

    Sort, fix i, then run opposite-end elimination on the remaining
    range. Duplicate i values are skipped up front; duplicate L/R values
    are ONLY skipped after a triplet is recorded (skipping them earlier
    could skip over a still-needed combination).
    """
    nums = sorted(nums)
    n = len(nums)
    result = []
    for i in range(n - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        if nums[i] > 0:
            break
        left, right = i + 1, n - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                result.append([nums[i], nums[left], nums[right]])
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1
                while left < right and nums[right] == nums[right + 1]:
                    right -= 1
        return result

```

Trace for **i** fixed at `nums[i] = -4` on `[-4, -1, -1, 0, 1, 2]`:

left	right	nums[L]	nums[R]	sum
1	5	-1	2	-3
2	5	-1	2	-3
3	5	0	2	-2
4	5	1	2	-1

6.3 The two duplicate-handling rules (both worth getting exactly right)

3Sum has **two separate** duplicate issues, and the source chat explicitly worked through a correction on the second one.

Rule 1 -- skip duplicate `i` values up front. If two different indices hold the same value, starting a full L/R search from both would just rediscover the same triplets. So: `if i > 0 and nums[i] == nums[i-1]: continue`.

Rule 2 -- duplicate L/R values are handled differently depending on timing.

A correction worked through live in the source chat

The question posed was: *why don't we skip duplicate L/R values before a valid triplet is found?* The first instinct answered "B -- because duplicates only matter after a solution has been recorded," which is incomplete. The precise answer is **A**: skipping early could skip over a still-needed combination.

Why skip duplicate L/R only AFTER recording a triplet?

BEFORE a solution is found

`[-2, -1, -1, 0, 3]`

L R

the two -1s are duplicate VALUES, but the pointer may still need to move THROUGH them while searching

-> skipping early could skip a valid combination

answer: A, not B

AFTER a triplet is recorded

`[-2, 0, 0, 2, 2]`

L R

`-2+0+2=0` recorded as `[-2,0,2]`; moving to the next 0 / next 2 would reproduce the SAME triplet

-> now duplicates are safe to skip

Before any triplet is found for the current `i`, a duplicate value may still be a necessary stepping stone -- the pointer has to pass through it while searching, so pre-emptively jumping over it could jump past the only valid combination. Only **after** a triplet is recorded does moving to the next occurrence of the same value guarantee reproducing that exact triplet again, which is when skipping becomes not just safe but necessary (to avoid duplicate output):

```
while left < right and nums[left] == nums[left - 1]:
    left += 1
while left < right and nums[right] == nums[right + 1]:
    right -= 1
```

Sanity check -- a triplet found by scanning **through** a duplicate value before any answer was recorded:

- **triplet_found_through_duplicate:** `[-2, -1, 3]`

6.4 3Sum Closest (LC 16)

Same sort + fix `i` + L/R scan, but instead of stopping at an exact match we track the best `abs(sum - target)` seen so far. The direction rule for moving `L / R` is unchanged -- it's still driven purely by whether the current sum is above or below `target`.

```
def three_sum_closest(nums: List[int], target: int) → int:
    """LC 16. 3Sum Closest.

    Same sort + fix i + L/R scan, but instead of stopping at equality we
    track the best abs(sum - target) seen, still moving L/R by the same
    monotonic proof (too small → L++, too large → R--).
    """
    nums = sorted(nums)
    n = len(nums)
    best = nums[0] + nums[1] + nums[2]
    for i in range(n - 2):
        left, right = i + 1, n - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if abs(total - target) < abs(best - target):
                best = total
            if total < target:
                left += 1
            elif total > target:
                right -= 1
            else:
                return target
    return best
```

6.5 3Sum Smaller (LC 259, premium)

Count triplets whose sum is **less than** `target`, rather than finding one exact match. The extra insight: once `nums[i] + nums[left] + nums[right] < target` is true, it stays true for `nums[left]` paired with **every** index between `left` and `right` (not just `right` itself), because the array is sorted -- so all of those `(right - left)` triplets can be counted in one step before advancing `left`.

```
def three_sum_smaller(nums: List[int], target: int) → int:
    """LC 259. 3Sum Smaller (premium).

    Count triplets with sum < target. Sort, fix i, then L/R: if the sum
    is already small enough, EVERY value between L and R paired with L
    also works (because the array is sorted, incrementing L while
    keeping R only increases the sum), so we count (right - left)
    triplets at once and advance L. Otherwise R must shrink.
    """
    nums = sorted(nums)
    n = len(nums)
    count = 0
    for i in range(n - 2):
        left, right = i + 1, n - 1
        while left < right:
            if nums[i] + nums[left] + nums[right] < target:
                count += right - left
                left += 1
            else:
                right -= 1
    return count
```

6.6 4Sum (LC 18)

Exactly 3Sum with **one more** fixed outer index. Fix **i** and **j**, then run the identical L/R elimination proof on what's left:

```

def four_sum(nums: List[int], target: int) → List[List[int]]:
    """LC 18. 4Sum.

    3Sum with one extra fixed pointer: fix i and j, then run the same
    L/R elimination on the rest.
    """
    nums = sorted(nums)
    n = len(nums)
    result = []
    for i in range(n - 3):
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        for j in range(i + 1, n - 2):
            if j > i + 1 and nums[j] == nums[j - 1]:
                continue
            left, right = j + 1, n - 1
            while left < right:
                total = nums[i] + nums[j] + nums[left] + nums[right]
                if total < target:
                    left += 1
                elif total > target:
                    right -= 1
                else:
                    result.append([nums[i], nums[j], nums[left], nums[right]])
                    left += 1
                    right -= 1
                    while left < right and nums[left] == nums[left - 1]:
                        left += 1
                    while left < right and nums[right] == nums[right + 1]:
                        right -= 1
            return result

```

Complexity: n^2 choices for (i, j) times an $O(n)$ two-pointer scan = $O(n^3)$. The important pattern is not the extra loop -- it's that k Sum is always "fix $k - 2$ indices, then fall back to the Two Sum II base case."

6.7 Valid Triangle Number (LC 611)

A less obvious k Sum-flavored problem: count triples that can form a triangle's side lengths. Sort, then fix the **largest** side `nums[k]` by scanning from the right, and run L/R on the prefix `[0, k-1]`. The triangle inequality only needs one check once the array is sorted (`nums[L] + nums[R] > nums[k]`) -- the other two inequalities hold automatically because `nums[L], nums[R] ≤ nums[k]`. If the inequality holds, it also holds for every index between `L` and `R` paired with `R`, so $(right - left)$ triangles can be counted at once.

```

def valid_triangle_number(nums: List[int]) → int:
    """LC 611. Valid Triangle Number.

    Sort, fix the LARGEST side at index k (scanning from the right),
    then use L/R on the prefix:  $nums[L] + nums[R] > nums[k]$  is the
    triangle inequality (the other two inequalities hold automatically
    once the array is sorted and  $R < k$ ). If it holds, every index
    between L and R also works with R, so count  $(R - L)$  at once and
    shrink R; otherwise L must grow.
    """
    nums = sorted(nums)
    n = len(nums)
    count = 0
    for k in range(n - 1, 1, -1):
        left, right = 0, k - 1
        while left < right:
            if nums[left] + nums[right] > nums[k]:
                count += right - left
                right -= 1
            else:
                left += 1
    return count

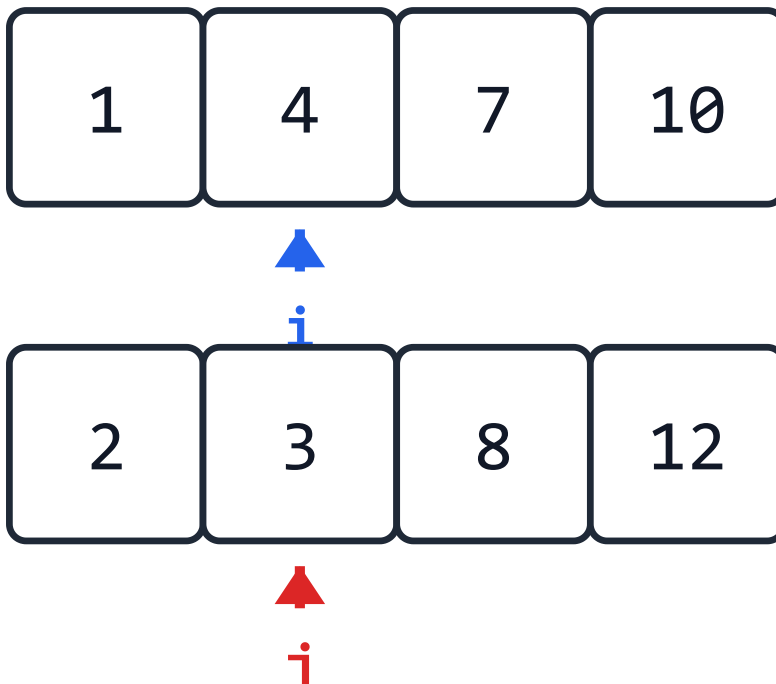
```

Part 7: Merge-Style Pointers -- One Per Sequence

7.1 The idea

Given two **sorted** sequences, keep one pointer per sequence. At every step, each pointer represents "the smallest unprocessed candidate from its own sequence." Whichever candidate is smaller is *guaranteed* to be the next element in the fully merged order -- nothing later in either array can be smaller than what's already at the front.

$A[i]=4$ vs $B[j]=3$ -> 3 is smaller,
consume it, $j++$



```
A[i] ≤ B[j] → A[i] is the smallest unprocessed value overall
              → consume it, i += 1
otherwise   → consume B[j], j += 1
```



```
def merge_sorted(a: List[int], b: List[int]) → List[int]:
    """General merge-style two pointers: each index represents the
    smallest unprocessed candidate of its own sequence, so whichever is
    smaller must be next in the merged order."""
    i = j = 0
    result = []
    while i < len(a) and j < len(b):
        if a[i] ≤ b[j]:
            result.append(a[i])
            i += 1
        else:
            result.append(b[j])
            j += 1
    result.extend(a[i:])
    result.extend(b[j:])
    return result
```

Complexity: $O(n + m)$ -- each pointer only ever moves forward through its own array.

7.2 Merge Sorted Array (LC 88)

The in-place LeetCode variant has a twist: `nums1` has trailing empty capacity to merge into, and if we scanned from the **front** we'd overwrite values in `nums1` before reading them. The fix is to scan **from the back**: at every step the *largest* unprocessed candidate goes into the back of the write region, which is the mirror image of the general merge above (front-scan + smallest-first becomes back-scan + largest-first).

```
def merge_sorted_array_inplace(
    nums1: List[int], m: int, nums2: List[int], n: int
) → None:
    """LC 88. Merge Sorted Array.

    Same merge-style idea, but nums1 has trailing empty space and we
    must not overwrite unread values, so pointers scan from the BACK: at
    every step the LARGEST unprocessed candidate goes at the back of the
    write region, which is the mirror image of the general merge above.
    """
    i, j, k = m - 1, n - 1, m + n - 1
    while j ≥ 0:
        if i ≥ 0 and nums1[i] > nums2[j]:
            nums1[k] = nums1[i]
            i -= 1
        else:
            nums1[k] = nums2[j]
            j -= 1
        k -= 1
```

Part 8: Hard Problems Requiring Their Own Proof

These problems are where recognition alone isn't enough -- each one needs a genuine, problem-specific proof before the pointer movement rule becomes obvious. The general question to ask throughout:

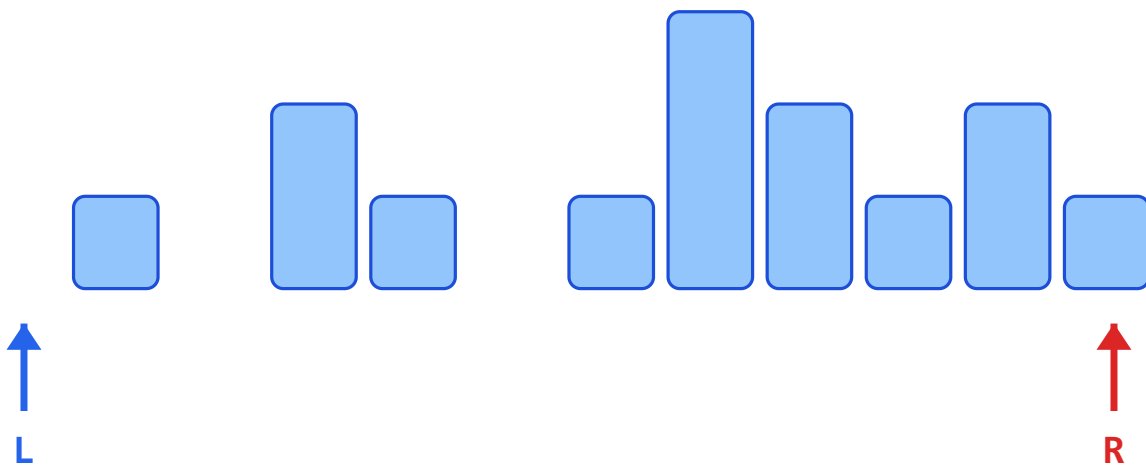
Don't ask merely *which* pointer can move. Ask *what information makes that movement safe*.

8.1 Trapping Rain Water (LC 42)

The water trapped at index i is $\min(\text{max_left}, \text{max_right}) - \text{height}[i]$, which normally suggests prefix/suffix max arrays. Two pointers get this down to $O(1)$ extra space with one key insight:

If $\text{left_max} \leq \text{right_max}$, the water level at left is determined entirely by left_max -- we don't even need to know exactly where the right-side boundary is, only that it's guaranteed to be at least left_max somewhere to the right.

$\text{left_max} \leq \text{right_max} \rightarrow \text{water}[L]$ is safely determined by left_max



```
def trap(height: List[int]) → int:
    """LC 42. Trapping Rain Water.

    Maintain left_max/right_max. If left_max ≤ right_max, the water
    level at 'left' is determined entirely by left_max, because the
    right side is GUARANTEED to have a wall at least that high somewhere
    -- we don't need to know exactly where. Process left and advance;
    symmetric on the other side otherwise.
    """
    left, right = 0, len(height) - 1
    left_max = right_max = 0
    water = 0
    while left < right:
        if height[left] ≤ height[right]:
            if height[left] ≥ left_max:
                left_max = height[left]
            else:
                water += left_max - height[left]
                left += 1
        else:
            if height[right] ≥ right_max:
                right_max = height[right]
            else:
                water += right_max - height[right]
                right -= 1
    return water
```

Trace on `[0,1,0,2,1,0,1,3,2,1,2,1]` (columns: which side processed, its index, its running max so far, cumulative water):

side	index	running_max	water_so_far
L	0	0	0
L	1	1	0
L	2	1	1
R	11	1	1
L	3	2	1
L	4	2	2
L	5	2	4
L	6	2	5
R	10	2	5
R	9	2	6
R	8	2	6

Total water trapped: 6 .

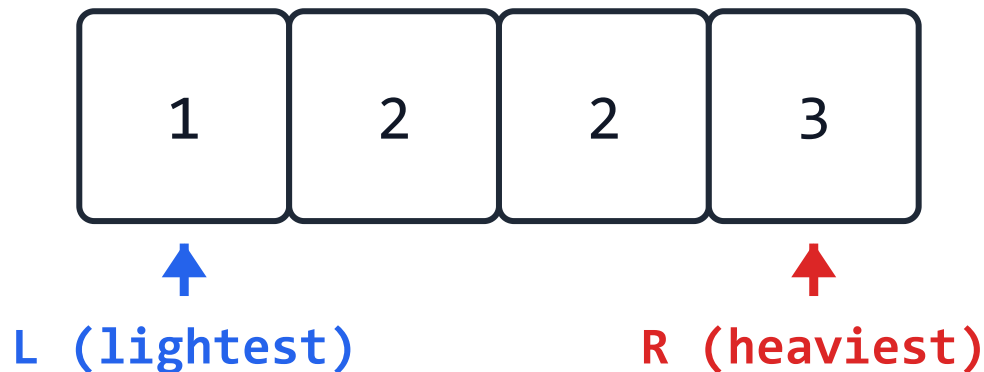
8.2 Sort + Greedy Endpoint Decisions

Several problems combine sorting with a *greedy* (not strictly elimination) endpoint decision. The two-pointer part is still doing the same job -- narrowing the search space by one provably-settled decision per step -- but the proof is a greedy exchange argument instead of a pure impossibility argument.

Boats to Save People (LC 881)

Each boat carries at most two people, capacity `limit` . Sort by weight. The **heaviest** remaining person must go on *some* boat -- try pairing them with the **lightest** remaining person. If that pair still fits, take both; if not, the heaviest person must go alone, because everyone else remaining is at least as heavy, so nobody else can pair with them either.

`limit=3: heaviest(3) must sail; try pairing
with L`



```
def num_rescue_boats(people: List[int], limit: int) → int:
    """LC 881. Boats to Save People.

    Sort. The heaviest remaining person MUST go on some boat; try
    pairing them with the lightest remaining person. If that pair still
    fits under `limit`, take both; otherwise the heaviest goes alone
    (everyone else is at least as heavy, so nobody else can pair with
    them either).
    """
    people = sorted(people)
    left, right = 0, len(people) - 1
    boats = 0
    while left ≤ right:
        if people[left] + people[right] ≤ limit:
            left += 1
        right -= 1
        boats += 1
    return boats
```

Trace on `people=[1,2,2,3]` , `limit=3` (columns: left after this step, right before decrement, whether this step paired two people):

left	right	paired?
0	3	False
1	2	True
1	1	False

Bag of Tokens (LC 948)

Sort the tokens. If we can afford the cheapest remaining token, playing it face up (spend `power` , gain a point) can only help future affordability, so it's always at least as good as not playing it. If we can't afford it, and we have a point to spend, playing the *most expensive* remaining token face down (spend a point, regain `power`) is the only way to buy affordability back for the cheap tokens -- but only worth doing if a point actually exists to spend and more than one token remains.

```

def bag_of_tokens(tokens: List[int], power: int) → int:
    """LC 948. Bag of Tokens.

    Sort. Greedy two-pointer: if we can afford the cheapest remaining
    token, play it face up (spend power, gain a score) to maximize how
    many more tokens we can eventually afford. If we can't, and there is
    a token left, play the most expensive one face down (spend a score
    to regain power) to buy affordability for the cheap tokens -- but
    only if we actually have a score to spend.
    """
    tokens = sorted(tokens)
    left, right = 0, len(tokens) - 1
    score = 0
    best = 0
    while left ≤ right:
        if power ≥ tokens[left]:
            power -= tokens[left]
            score += 1
            best = max(best, score)
            left += 1
        elif score > 0 and left < right:
            power += tokens[right]
            score -= 1
            right -= 1
        else:
            break
    return best

```

Meeting Scheduler (LC 1229, premium)

Sort both people's slot lists, then merge-style walk them. The overlap of the current pair of slots is `[max(start1, start2), min(end1, end2)]`. If that overlap is at least `duration` long, return it immediately. Otherwise, the slot that **ends earlier** can never overlap anything that comes after the other slot's start either (its own end is already the tightest constraint), so it is eliminated and only that pointer advances.

```
def meeting_scheduler(
    slots1: List[List[int]], slots2: List[List[int]], duration: int
) → List[int]:
    """LC 1229. Meeting Scheduler (premium).

    Sort both slot lists, then walk them merge-style: the overlap of the
    current pair is [max(start1, start2), min(end1, end2)]. If that
    overlap is long enough, return it. Otherwise the slot that ends
    EARLIER can never overlap anything after the other slot's start
    either, so it is eliminated and we advance only that pointer.
    """
    slots1 = sorted(slots1)
    slots2 = sorted(slots2)
    i = j = 0
    while i < len(slots1) and j < len(slots2):
        lo = max(slots1[i][0], slots2[j][0])
        hi = min(slots1[i][1], slots2[j][1])
        if hi - lo ≥ duration:
            return [lo, lo + duration]
        if slots1[i][1] < slots2[j][1]:
            i += 1
        else:
            j += 1
    return []
```

8.3 A General Partition Pattern

The source chat's final practice problem generalizes Sort Colors and Move Zeroes to a plain two-region partition (negative values on one side, non-negative on the other), using opposite-end pointers instead of a slow/fast scan:

```
left side → already satisfies the condition
right side → already satisfies the opposite condition
middle → unknown
```

```
def partition_negatives(nums: List[int]) → None:
    """General partition pattern from the source chat (negative | non-
    negative), the same in-place-partition idea that Sort Colors and
    Move Zeroes generalize to two regions instead of one boundary."""
    left, right = 0, len(nums) - 1
    while left ≤ right:
        while left ≤ right and nums[left] < 0:
            left += 1
        while left ≤ right and nums[right] ≥ 0:
            right -= 1
        if left < right:
            nums[left], nums[right] = nums[right], nums[left]
            left += 1
            right -= 1
```

Part 9: Recognition -- Which Pattern, or None At All

9.1 The pattern map

TWO POINTERS

OPPOSITE ENDS

sorted pair / target sum
palindrome
container / area
3Sum / 4Sum (+fix i,j)

SAME DIRECTION

compact / filter
partition regions
dedupe in place
move valid to front

MULTI-SEQUENCE

merge two sorted
subsequence scan
one ptr per array

+ FAST/SLOW (linked-list speed) and FIXED-GAP (distance k)

The universal test:

Can moving THIS pointer be PROVEN to eliminate possibilities? If not, two pointers is the wrong tool.

9.2 Recognition cheat sheet

Problem signal	Pattern to think of
Sorted array + find pair with target sum	Opposite ends L/R
Sorted array + pair condition	Opposite ends
Sorted array + fix one element + find pair	Fix <code>i</code> + L/R
3Sum / 4Sum-style problem	Sort + fixed pointer(s) + L/R
Maximize/minimize using two endpoints	Opposite ends
Compare characters/elements from both ends	Opposite ends
Palindrome	Opposite ends
Remove/filter elements in-place	Same-direction slow/fast
Compress/compact an array in-place	Same-direction slow/fast
Move valid elements to the front	Same-direction slow/fast
Move zeroes / remove duplicates / remove val	Same-direction slow/fast

Problem signal	Pattern to think of
Two sorted arrays/sequences	Merge-style, one pointer each
Find something k positions apart	Fixed-gap pointers
Linked-list middle/cycle	Fast/slow pointers
Compare a subsequence against another string	Same-direction string ptrs

9.3 Fast/slow and fixed-gap: recognition only (full coverage lives in Linked List)

Two more variants round out the family, both usually seen on linked lists:

- **Fast/slow (different speeds):** `slow` advances one node per step, `fast` advances two. Used for finding a linked list's middle and for Floyd's cycle detection. The recognition signal is "linked-list midpoint/cycle."
- **Fixed-gap:** keep two pointers a constant distance `k` apart and advance them together. The recognition signal is *"find the element that is `k` positions behind/ahead of another element."*

This guide's source chat covered both **only at the recognition level** -- the derivations, proofs, and code already live in the **Linked List** guide, and are deliberately not duplicated here.

9.4 The 3 questions to ask on a new problem

1. Do I have **two meaningful positions**?
2. Is there some **ordering / monotonic property** I can exploit?
3. Can moving a pointer **safely eliminate possibilities**?

If the answer to all three is yes, two pointers is a strong candidate.

9.5 When NOT to use two pointers

When NOT to reach for two pointers

Unsorted + target lookup	HashMap
Arbitrary previously-seen values	Hashing / set
Contiguous subarray, maintain validity	Sliding Window
Pointer move can't PROVE elimination	wrong tool

The crucial distinction, stated directly by the source material:

Two pointers isn't about having two variables called `left` and `right`. It's about having a reason that moving one pointer eliminates possibilities.

If a problem is unsorted with no exploitable monotonic structure (classic "two numbers sum to target" on an *unsorted* array, for example), sorting would destroy the original indices you need to return -- reach for a HashMap instead. If the problem is about a **contiguous subarray** whose validity needs to be maintained as elements enter/leave a window, that's Sliding Window, not this family.

Part 10: The Universal Thought Process

For any new two-pointer candidate, run this checklist in order:

1. What are the two positions **representing**?
2. What does each pointer **mean**, concretely, in terms of the problem's state?
3. What is my **invariant** -- the fact that stays true at every step?
4. What happens if I move the left/lower pointer?
5. What happens if I move the right/higher pointer?
6. Can I **prove** that one movement eliminates possibilities that cannot contain the answer?
7. Does each pointer move only forward/inward, never backward? If so, an $O(n)$ -ish scan is likely.

Questions 6 and 7 matter most. The source material is explicit that the goal is proving pointer movement, not pattern-matching a memorized template.

Complexity mental model

Shape	Typical complexity
One pointer across n , another across n	$O(n)$
Sort + two pointers	$O(n \log n) + O(n) = O(n \log n)$
3Sum (fix 1 index + L/R scan)	n choices $\times$ n scan = $O(n^2)$
4Sum (fix 2 indices + L/R scan)	n^2 choices $\times$ n scan = $O(n^3)$

The general rule: every index that gets "fixed" by an outer loop before falling back to an opposite-end L/R scan multiplies the complexity by another factor of n ; the L/R scan itself is always $O(n)$ because each of L and R only ever moves in one direction.

Part 11: Interview Recognition Drills

Each of these mirrors a question posed live in the source dialogue. Answer before revealing the reasoning underneath -- that's the exercise.

Q1. `nums = [2, 4, 5, 8, 12, 15]`, `target = 14`. `L=0`, `R=5`: `2 + 15 = 17 > 14`. Which pointer moves, and why in one sentence?

Move `R`. `2` is already the smallest available value, and even `2 + 15 > 14`, so `15` cannot pair with anything remaining -- it's eliminated.

Q2. `nums = [1, 3, 5, 7, 10]`, `target = 11`. `L=0`, `R=4`: `1 + 10 = 11`. What do we do, and why don't we move either pointer?

Return immediately. We've already found a valid pair; there's nothing left to eliminate or search for.

Q3. `height[L] = height[R]` in Container With Most Water. Which pointer should move?

Either one is safe. The current pair's area can never be reused at a better width, and both sides are equally limiting, so there's no reason to preserve one over the other.

Q4. Given an *unsorted* array, find two numbers summing to `target`. Would you sort it and use opposite-end pointers?

No -- sorting destroys the original indices, which most versions of this problem require you to return. Use a HashMap instead ($O(n)$ time, $O(n)$ space, one pass, no ordering assumption needed).

Q5. In 3Sum, why do we skip duplicate `L / R` values only *after* recording a triplet, not before?

Before a solution is found, a duplicate value may still be a necessary stepping stone the pointer must pass through while searching -- skipping it early could skip a still-needed combination. After a solution is recorded, moving to the next occurrence of the same value is guaranteed to reproduce the same triplet, so skipping becomes both safe and necessary.

Q6. In Sort Colors, why doesn't `mid` advance after swapping in a `2` from `high` ?

The value just swapped in from `high` has never been examined -- it could be a `0` , `1` , or another `2` . Advancing `mid` would skip checking it.

Q7. Why does sorting matter for two-pointer elimination proofs in general?

Sorting creates a monotonic relationship: moving `left` `right` can only increase `nums[left]` , and moving `right` `left` can only decrease `nums[right]` . That monotonic guarantee is what turns "the sum is too small/large" into a valid proof that a pointer's current value is impossible -- without it, a valid answer could be sitting anywhere.

Q8. A problem says "find the element that is `k` positions behind/ahead of another element." What relationship should come to mind immediately?

Same-direction pointers held a fixed gap of `k` apart, advanced together.

Q9. Merge-style pointers: `A[i] = 4` , `B[j] = 2` . Which pointer moves, and why is it safe?

Move `j` . Because `B` is sorted, nothing after `B[j]` can be smaller than `2` , and `A[i]` is already `4` -- so `2` is guaranteed to be the next element in the merged order.

Q10. True or false: two pointers is always $O(n)$.

False. A single opposite-end or same-direction scan is $O(n)$, but patterns that fix outer indices before falling back to that scan (3Sum, 4Sum) multiply the complexity by a factor of `n` per fixed index -- 3Sum is $O(n^2)$, 4Sum is $O(n^3)$.

Closing thought

Every pattern in this guide -- opposite ends, same-direction slow/fast, merge-style, sort + fixed pointer(s) -- reduces to the same underlying question the source material returns to again and again:

What does each pointer represent, and why is it safe to move it?

Templates memorized without that proof break the moment a problem's details shift slightly (as the two corrections in Parts 1 and 6 show directly). Templates *derived* from the elimination proof generalize to problems you've never seen before.